

Exercício 2 - cap 4

Discente: Raphael Ramos da Silva

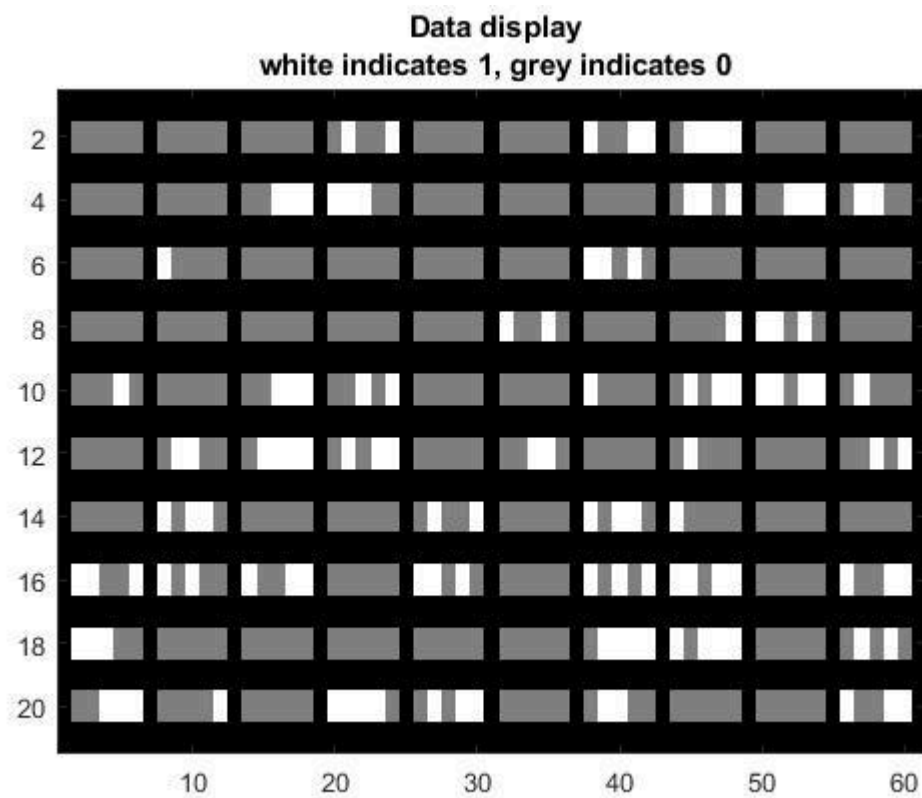
Classificação binária - Executar o código MATLAB NNode1.m (anexo) para obter os resultados semelhantes aos vistos nas Figuras 4.13, 4.15, 4.17 e 4.18 do livro.

- Validação do gradiente - Executar o código NNode test.m (será anexado) para gerar a Figura 4.26.
- Classificação multiclasse - Executar o código NNode multiclass.m(será anexado) para obter resultados semelhantes ao Exemplo 4.4.2.
- Entregar tudo na forma de um notebook (Python ou Matlab) com discussão dos resultados.
- Exercício descrito na seção 4.8 do livro.
- Link com descrições do laboratório: Lab NN

Classificação binária (NNode1.m)

Data display

Os dados de entrada utilizados para o treinamento das NN (*Neural Networks*) deste exercício estão ilustrados a seguir.



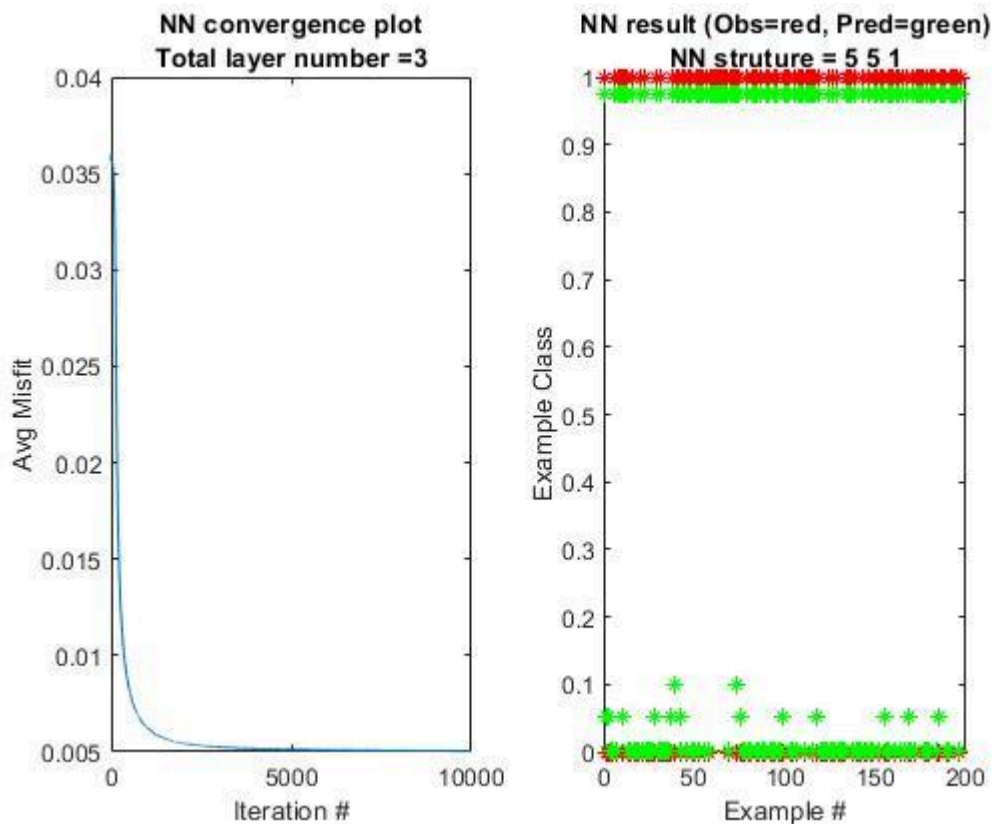
Ao todo são $M=100$ vetores $x_{5,1}$ de entrada - os quais foram dispostos horizontalmente na figura acima em um grid 10x10 - cada um com $N=5$ linhas (features) que podem assumir valor 1 (branco) ou 0 (cinza). Cada vetor $x_{5,1}$ do conjunto de dados é rotulado como 1 ou 0. Então, temos um problema de classificação binária.

Na classificação, a Rede Neural (*Neural Network - NN*) recebe um vetor de $N=5$ elementos e classifica-o 0 ou 1. Sobre sua estrutura, ela pode ser configurada para ser treinada com n camadas ocultas, cada uma podendo alocar $k^{[i]}$ nós (*nodes*), em que i é o número da camada.

Então, se o programa `NNode.m` recebeu `nodes = [5 1]` como entrada, temos na verdade uma estrutura [5 5 1], em que a primeira posição representa o vetor de entrada de $k^{[0]} = 5$ nós/elementos; logo após, a camada oculta com $k^{[1]} = 5$ nós; e, por fim, a camada de saída, com 1 nó.

Figura 4.13

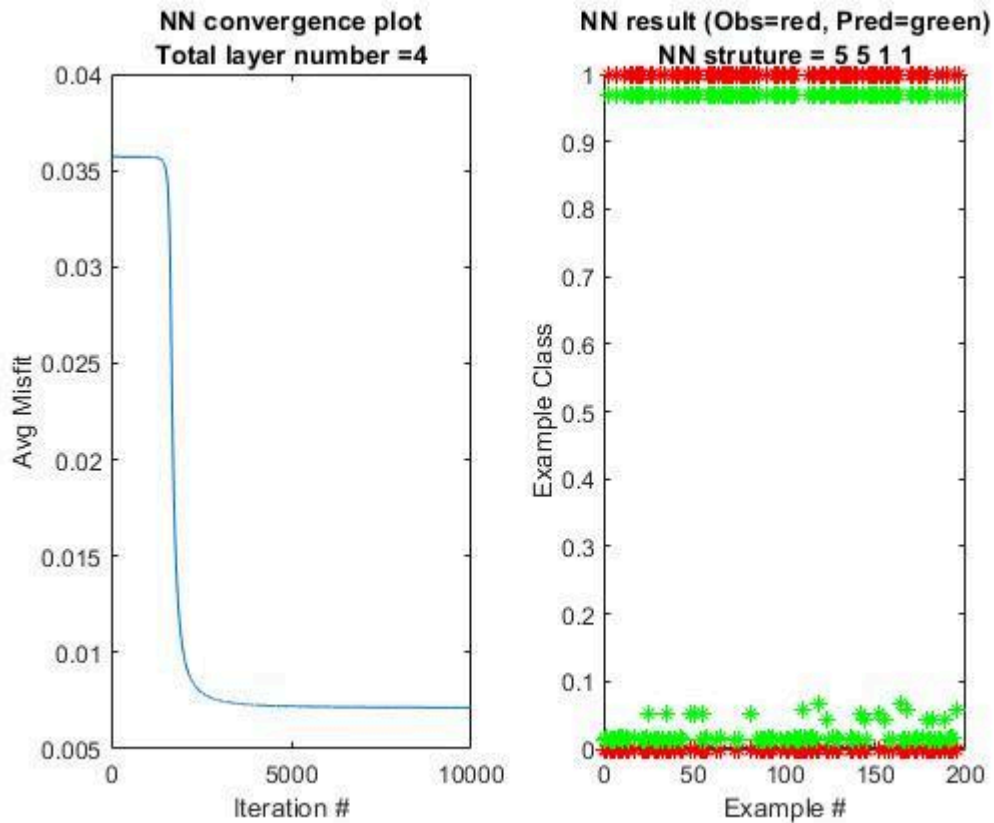
Os parâmetros de entrada foram: `nodes = [5 1]`, norma L2 (1) e sigmoid como função de ativação (1).



No primeiro gráfico, a perda cai de forma suave e rápida desde as primeiras iterações, estabilizando assintoticamente em 0.005. Com relação a classificação, a rede aprende bem a separação entre as classes, porque a maioria das amostras do rótulo 0 fica realmente próxima de 0, e as do rótulo 1 ficam em torno de 0.97. Algumas poucas amostras da classe 0 apresentam divergências razoáveis (vaores em torno de 0.05-0.1), mas estas podemos classificar como dispersos, podendo ser ignorados se fosse o caso de definir um limiar (*threshold*) para classificá-las exatamente como 0.

Figura 4.15

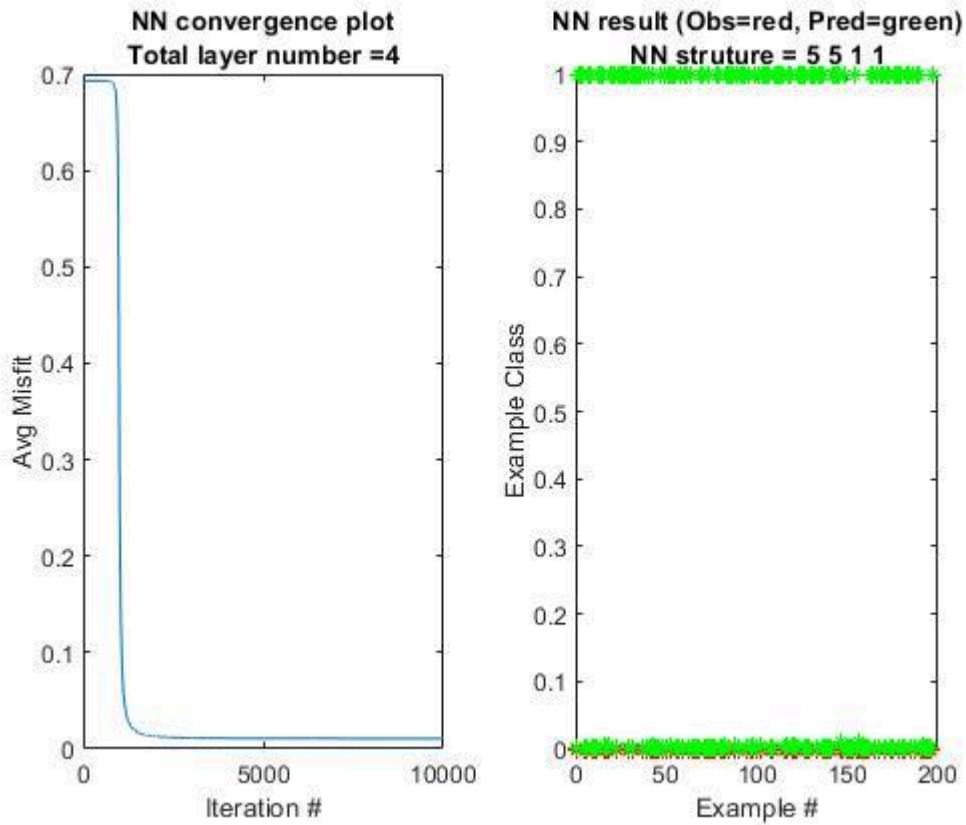
Os parâmetros de entrada foram: `nodes = [5 1 1]`, norma L2 (1) e sigmoid como função de ativação (1).



Com mais uma camada oculta adicionada, os resultados mostram que, no gráfico de erro, há presença de um *plateau* inicial até cerca de 1500 épocas no treinamento, antes de cair abruptamente e estabilizar em 0.0075. Apesar desse patamar, a inferência mostra que aquelas classificações dispersas entre 0.05-0.1 agora estão concentradas abaixo de 0.1, demonstrando que a rede conseguiu capturar nuances dos dados que antes gerava em pequenas variações.

Figura 4.17

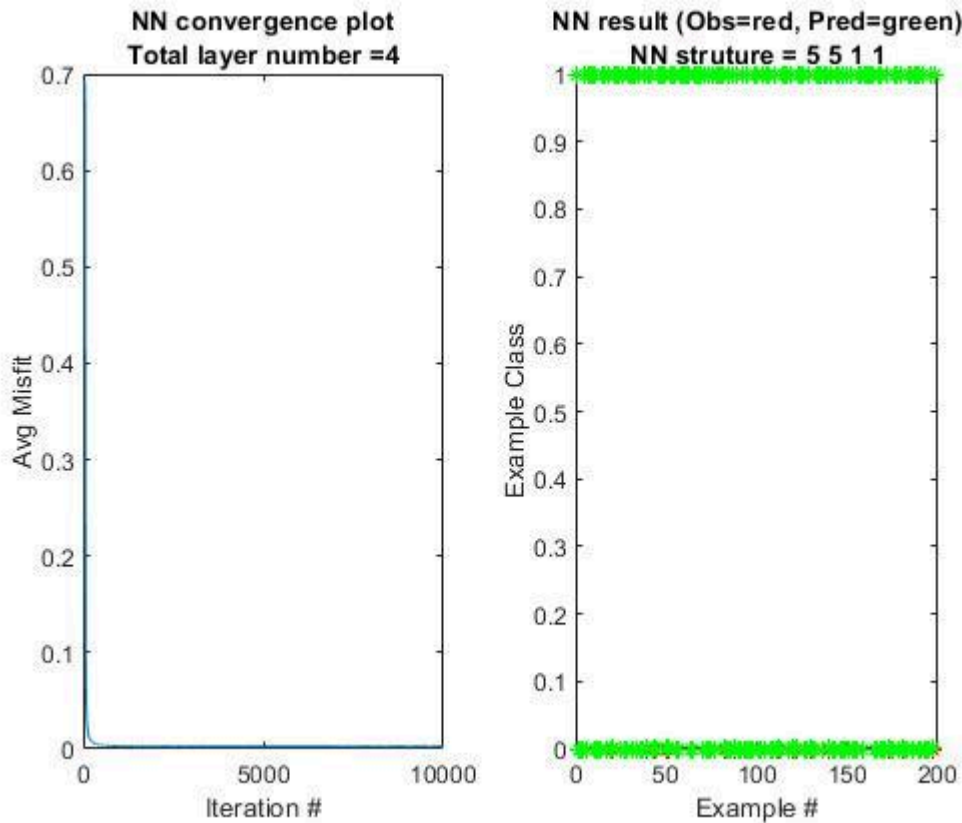
Os parâmetros de entrada foram: nodes = [5 1 1], entropia cruzada (2) e sigmoid como função de ativação (1).



Neste experimento, o *Avg Misfit* inicia em um patamar alto (≈ 0.7) e cai para próximo de 0.01. Em comparação com os experimentos anteriores que utilizaram a minimização pela norma L2, observou-se uma convergência relativamente rápida; contudo, a rede estagnou em um mínimo local, resultando em um colapso de classificação no qual a maioria dos exemplos foi atribuída à classe 1.

Figura 4.18

Os parâmetros de entrada foram: nodes = [5 1 1], entropia cruzada (2) e ReLU como função de ativação (1).



Diferentemente do experimento da Figura 4.17, a convergência do erro acontece imediatamente na primeira iteração: note como houve uma queda acelerada de 0.7 para próximo de 0. Consequentemente, o problema do *plateau* (patamar) apresentado nos experimentos das outras figuras, desaparece. Sobre a inferência, a rede separa de forma limpa ambas as classes (0 e 1).

O ajuste rápido dos pesos com ausência do *plateau* muito se deve ao fato da ReLU, em combinação com a minimização da entropia cruzada, evitar que os sinais de erro retropropagados se tornem extremamente pequenos ou nulos. Em outras palavras, evita-se que ocorra a saturação de gradientes nas camadas ocultas.

Essa saturação faz com que o algoritmo pare de atualizar os pesos das camadas mais profundas, impedindo ou desacelerando a convergência do modelo (resultando nos *plateaus* observados nos resultados desse exercício).

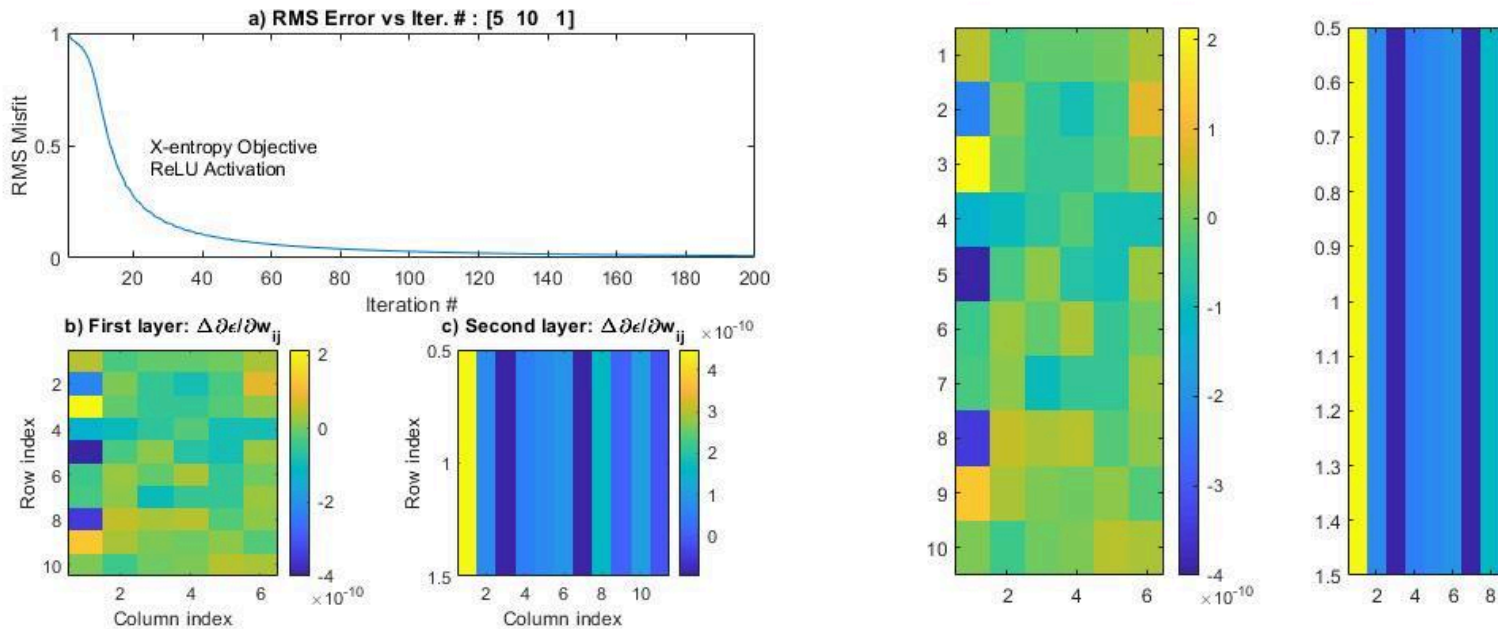
Classificação binária (NNode_test.m)

O gráfico da primeira linha mostra a evolução do erro *Root Mean Square* (RMS), onde para um conjunto de N dados de treino e diferenças $\epsilon_i = t_{\text{obs}} - t_{\text{pred}}$, temos:

$$\text{RMS} = \sqrt{\frac{1}{M} \sum_{i=1}^M \epsilon_i^2}$$

Os *plots b)* e *c)* mostram o estado dos gradientes calculados na iteração 200 (última iteração do treinamento), em que: a primeira camada mapeia 6 nós de entrada (5 features + 1 *bias*) para cada um dos 10 nós da segunda camada; e a segunda camada mapeia 11 nós da camada oculta (10 nós + 1 *bias*) para 1 nó da camada de saída. Matricialmente, portanto, os pesos dessa rede neural podem ser representados por uma matriz

$$W = \begin{bmatrix} W_{10 \times 6}^{[0]} & W_{1 \times 11}^{[1]} \end{bmatrix}$$



O erro RMS convergiu assintoticamente para 0 a partir da iteração 100, indicando que a rede conseguiu aprender bem os padrões dos dados de treino.

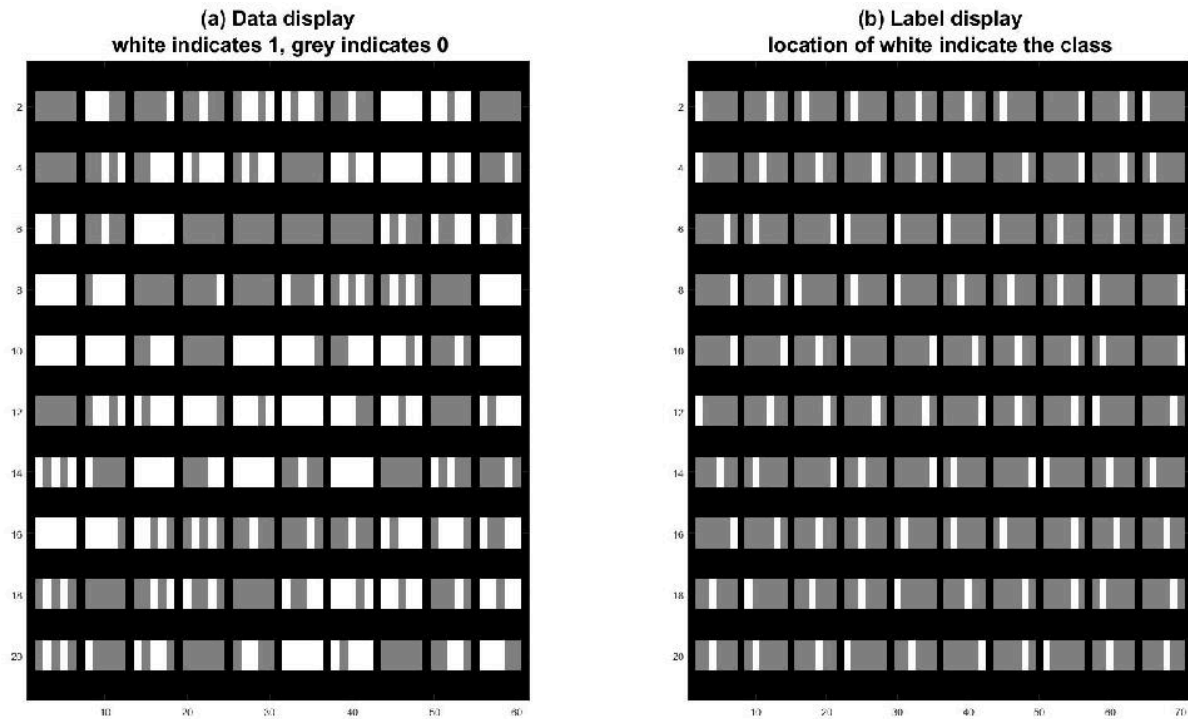
Na primeira camada, boa parte dos gradientes retropropagados se tornaram pequenos ($\approx 1 \times 10^{-10}$), dadas os tons de amarelo esverdeado serem mais predominante, sinalizando que os pesos atingiram um ponto de mínimo e praticamente pararam de atualizar.

Na camada de saída, por outro lado, temos gradientes ligeiramente maiores do que na primeira camada. Isso é justificado pelo *back-propagation*, que causa atenuação do erro (ϵ_{in}) à medida que é multiplicado pelas derivadas da regra da cadeia rumo às camadas mais profundas/iniciais.

Classificação binária (NNode_multiclass.m)

Data e Label display

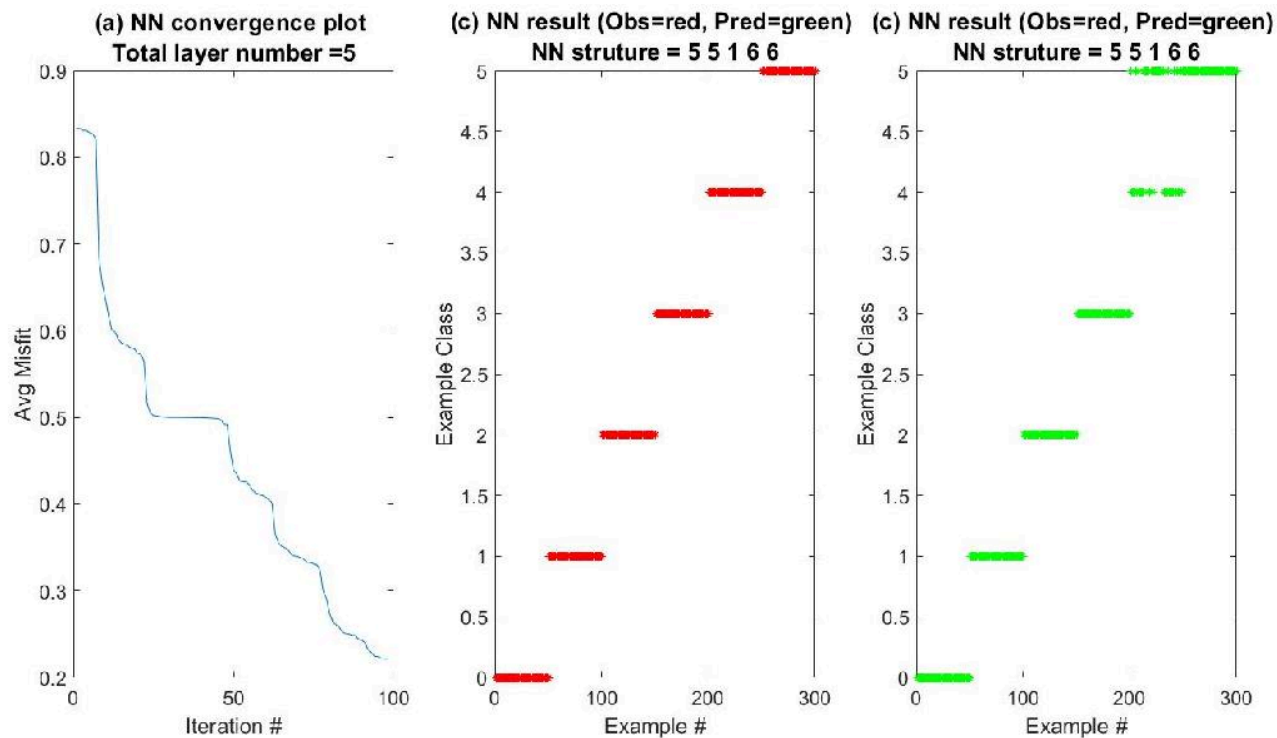
Os dados de entrada e os seus rótulos utilizados para o treinamento das NN (*Neural Networks*) deste exercício estão ilustrados a seguir.



Ao todo são $M=300$ vetores $x_{5,1}$ de entrada, cada um com $N=5$ linhas (features) - destes, apenas 100 foram dispostos horizontalmente na figura **(a)** em um grid 10x10 - que podem ser classificados como 0, 1, 2, 3, 4 ou 5. Visualmente, na figura **(b)**, essas classes estão posicionadas como barras brancas verticais brancas nas caixas cinzas. Então, temos um problema de classificação multiclasse.

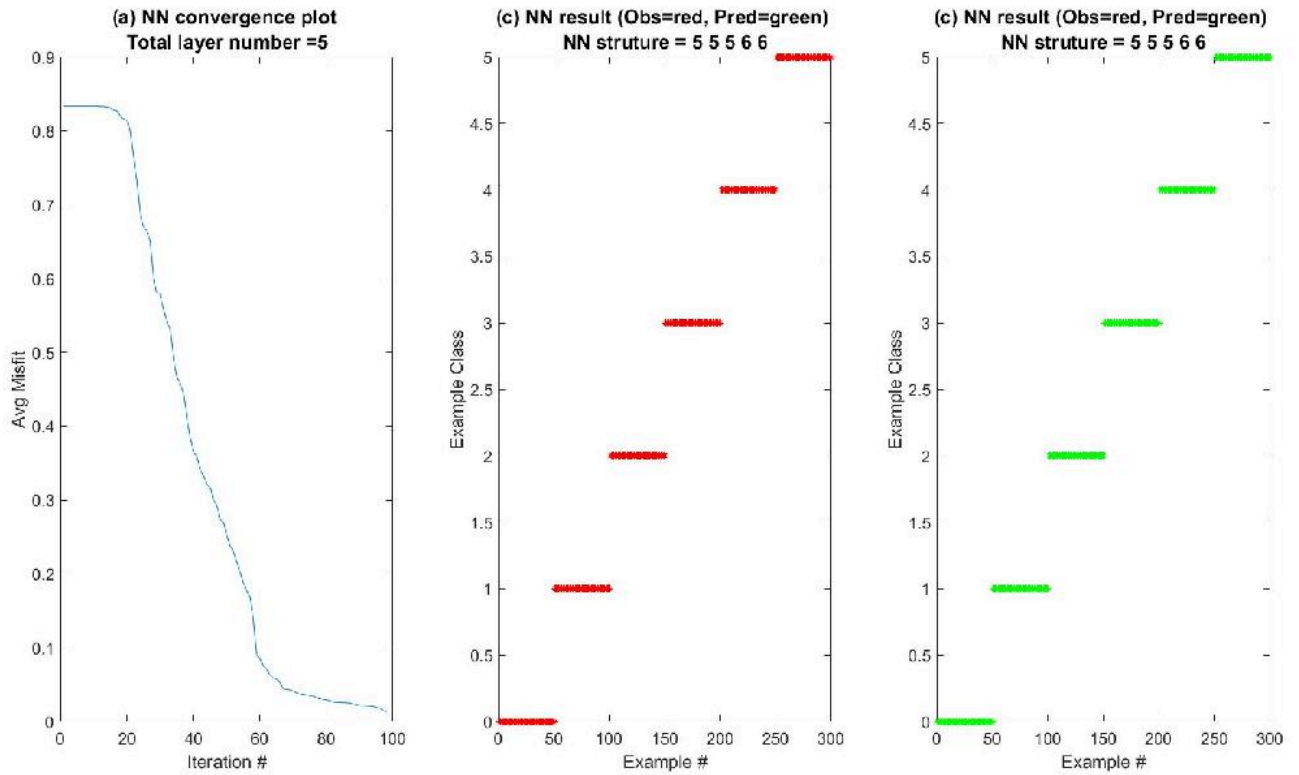
Sigmoid

Os parâmetros de entrada foram: nodes = [5 1 6], norma L2 (1) e sigmoid como função de ativação (1).



O erro médio inicia próximo de 0.83 e decresce lentamente com degraus de estagnação. Após 100 iterações, o erro estagna em um nível relativamente elevado (≈ 0.22). As classes 4 e 5 sofrem gravemente de classificações incorretas, quando comparadas as outras camadas.

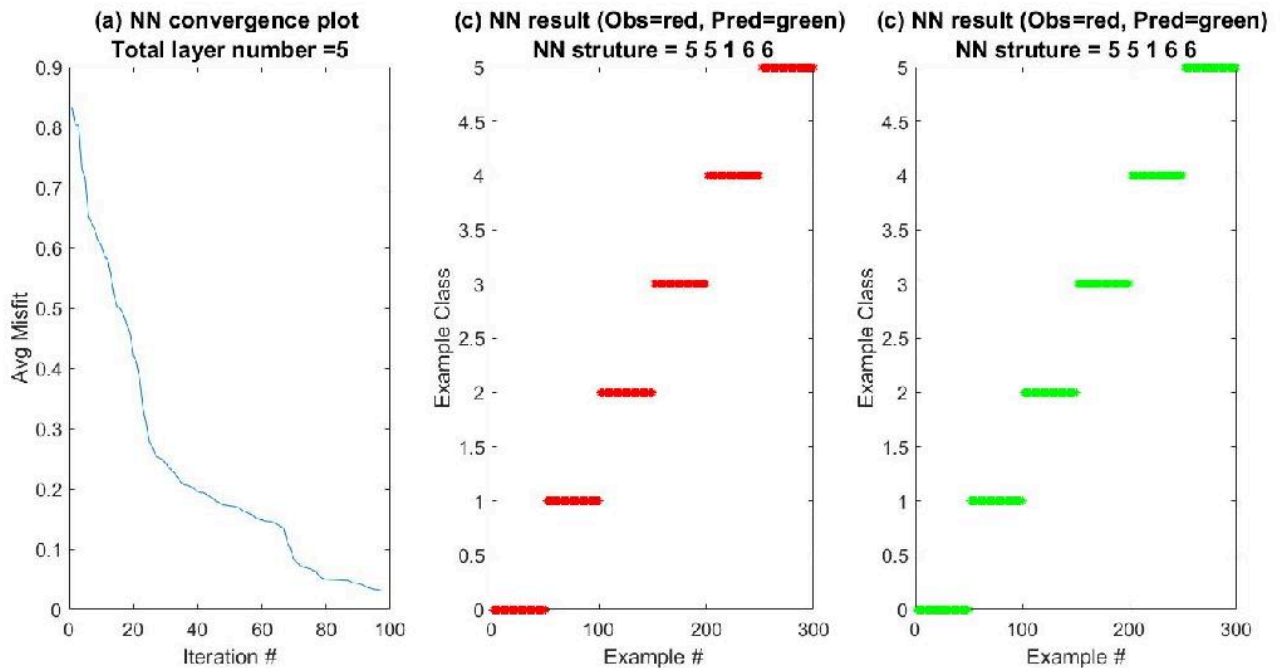
Observando que há estagnação do erro, muito provavelmente por conta do desaparecimento/saturação dos gradientes causados pelo gargalo da camada oculta de apenas um nó, aumentamos o número de nós nesta camada para 5, de modo que a arquitetura agora seja [5 5 6]. Com isso, obtemos os seguintes resultados.



Desta vez, temos uma convergência mais rápida na minimização da média das diferenças para um valor assintótico mais aceitável perto de 0.0. Além do mais, a rede melhorou sua precisão de classificação, visto que há um alinhamento praticamente perfeito entre as saídas preditas (em verde) e os rótulos observados (em vermelho) para todas as 6 classes (0 a 5).

ReLU

Os parâmetros de entrada foram: nodes = [5 1 6], norma L2 (1) e ReLU como função de ativação (1).



O erro médio inicia em torno de 0.85 e sofre uma queda contínua e acentuada. Ao chegar na iteração 100, o erro reduz-se a valores próximos de zero. Em relação a precisão de inferência, o alinhamento entre as classes reais e as previstas é praticamente perfeito para todas as 6 classes.

Em contraste da função sigmoide, que sofre com a saturação nas extremidades ($\sigma(z)'$ tende assintoticamente a 0.0 para pontos suficientemente distantes de $z_0 = 0.0$), a função ReLU possui derivada constante e igual a 1 para entradas positivas. Isso elimina a saturação do gradiente, permitindo que os sinais de erro retropropagados continuem fortes e atualizem a rede. Logo, aumentar o número de nós em alguma camada oculta nesta arquitetura é desnecessário.

Apesar disso, ao permitir que esta arquitetura tenha mais camadas, como no experimento a seguir em que a configuração muda apenas para [5 1 6 6 6], nota-se que há necessidade de um maior número de iterações para a convergência, visto que o custo médio ainda se encontra muito próximo a 0.1 na iteração 100, quando comparado ao experimento anterior, o qual chegou perto de 0.1 mais rapidamente.

